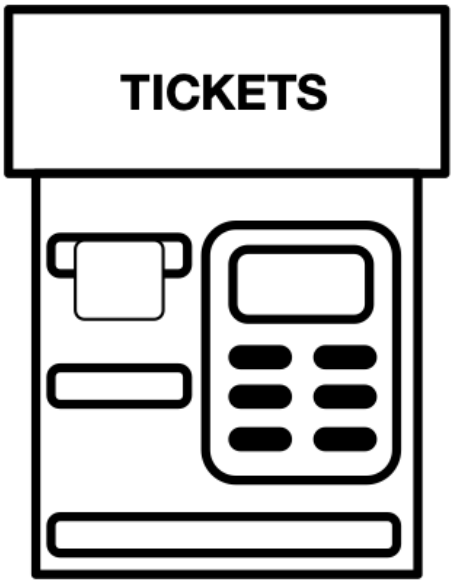
instituto
superior de
engenharia
de lisboa

Licenciatura em Engenharia Informática e de Computadores




# MÁQUINA DE VENDA DE BILHETES
# (*Ticket Machine*)

Projeto
de
Laboratório de Informática e Computadores
2025 / 2026 verão

publicado: 18 de fevereiro de 2026

# Conteúdo

**1 Descrição** **1**

**2 Arquitetura do Sistema** **3**

2.1 *Keyboard Reader* . . . . . 3

2.1.1 *Key Decode* . . . . . 3

2.1.2 *Ring Buffer* . . . . . 5

2.1.3 *Key Transmitter* . . . . . 6

2.2 *Coin Acceptor* . . . . . 6

2.3 *Port Expander LCD* . . . . . 7

2.4 *Port Expander Ticket Dispenser* . . . . . 8

2.5 *Control* . . . . . 9

**3 Calendarização** **12**

# Lista de Figuras


1 Diagrama de blocos da Máquina de Venda de Bilhetes (*Ticket Machine*) . . . . . 1
2 Arquitetura do sistema que implementa a máquina de venda de bilhetes (*Ticket Machine*) . . . . . 3
3 Diagrama de blocos do *Keyboard Reader* . . . . . 4
4 Bloco *Key Decode* . . . . . 4
a Diagrama de blocos . . . . . 4
b Diagrama temporal . . . . . 4
5 Diagrama de blocos do bloco *Key Scan* . . . . . 5
a versão I . . . . . 5
b versão II . . . . . 5
c versão III . . . . . 5
6 Diagrama de blocos do *Ring Buffer* . . . . . 5
7 Protocolo de comunicação com o módulo *Key Transmitter* . . . . . 6
8 Bloco *Coin Acceptor* . . . . . 6
a Diagrama de blocos . . . . . 6
b Diagrama temporal . . . . . 6
9 Diagrama de blocos do módulo *Port Expander LCD* (*PELCD*) . . . . . 7
10 Protocolo de comunicação com o módulo *Port Expander LCD* (*PELCD*) . . . . . 7
11 Diagrama de blocos do bloco *Serial Receiver* . . . . . 8
12 Diagrama de blocos do módulo *Port Expander Ticket Dispenser* (*PETD*) . . . . 8
13 Protocolo de comunicação com o *Port Expander Ticket Dispenser* (*PETD*) . . . 8
14 Protocolo de comunicação com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*) . . . . . 9
15 Diagrama lógico da Máquina de Venda de Bilhetes (*Ticket Machine*) . . . . . 9
16 Diagrama de *Gantt* relativo à calendarização do projeto) . . . . . 12


# Lista de Tabelas


1 Períodos do mecanismo de repetição . . . . . 4
2 Codificação do valor facial das moedas . . . . . 7


# Lista de Algoritmos


1 HAL - Hardware Abstract Layer . . . . . 10
2 KBD - Keyboard . . . . . 10
3 LCD . . . . . 10
4 Serial Emitter . . . . . 11
5 Ticket Dispenser . . . . . 11

# 1 Descrição

Pretende-se implementar uma máquina de venda de bilhetes (*Ticket Machine*), que permite a aquisição de bilhetes de comboio. O percurso é definido pela estação de origem, local de compra do bilhete e pela seleção do destino digitando o identificador da estação ou através das teclas ↑ e ↓, sendo exibido no ecrã além do identificador da estação de destino o preço e o tipo de bilhete (ida ou ida/volta). A ordem de aquisição é dada através da pressão da tecla de confirmação, sendo impressa uma unidade do bilhete exibido no ecrã. A máquina não realiza trocos e só aceita moedas de: 0,05€; 0,10€; 0,20€; 0,50€; 1,00€; e 2,00€. Para além do modo de Dispensa, o sistema tem mais um modo de funcionamento designado por Manutenção, que é ativado por uma chave de manutenção. Este modo permite o teste da máquina de venda de bilhetes, bem como permite iniciar e consultar os contadores de bilhetes e moedas. A máquina de venda de bilhetes é constituída pelo sistema de gestão (designado por *Control* na Figura 1) e pelos seguintes periféricos: um teclado de 16 teclas, um moedeiro (designado por *Coin Acceptor*), um ecrã *Liquid Cristal Display* (LCD) de duas linhas de 16 caracteres, um mecanismo de dispensa de bilhetes (designado por *Ticket Dispenser*) e uma chave de manutenção (designada por *M*) que define se a máquina de venda de bilhetes está em modo de Manutenção, conforme o diagrama de blocos apresentado na Figura 1.

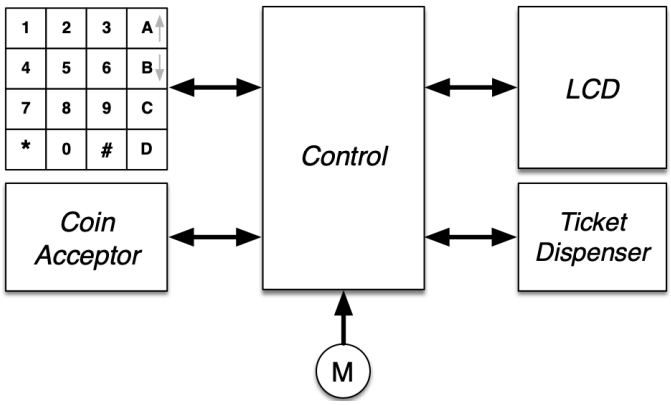



**Figura 1:** Diagrama de blocos da Máquina de Venda de Bilhetes (*Ticket Machine*)

Sobre o sistema podem-se realizar as seguintes ações em modo Venda:

- **Consulta e venda** A consulta de um bilhete é realizada digitando o identificador da estação de destino ou listando-a através das teclas ↑ e ↓. O processo de compra do bilhete inicia-se premindo a tecla '#'. Durante a inserção do respetivo valor monetário, é possível alterar o tipo de bilhete (ida ou ida/volta) premindo a tecla '∗', com a consequente alteração do preço da viagem afixado no *LCD*, duplicando o valor no caso de ida/volta. Durante a compra ficam afixados no *LCD* as informações referentes ao bilhete pretendido até que o mecanismo de impressão de bilhetes confirme que a impressão já foi realizada e a recolha do bilhete efetuada. A compra pode ser cancelada premindo a tecla '#', devolvendo as moedas inseridas.

Sobre o sistema podem-se realizar as seguintes ações em modo Manutenção:

- **Teste** – Esta opção do menu permite realizar um procedimento de consulta e venda de um bilhete, sem introdução de moedas e sem esta operação ser contabilizada como uma aquisição.
- **Consulta Bilhetes** – Premindo a tecla 'A', em seguida através das teclas ↑ e ↓, visualizam os contadores do número de bilhetes vendidos.
- **Consulta Moedas** – Premindo a tecla 'B', em seguida através das teclas ↑ e ↓, visualizam os contadores das moedas no cofre.
- **Iniciar os contadores de moedas e bilhetes** – Premindo a tecla 'C' e em seguida a tecla '∗', o sistema de gestão coloca os contadores de moedas e bilhetes a zero, iniciando um novo ciclo de contagem.
- **Desligar** – Premindo a tecla 'D' permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores de moedas e da lista de bilhetes vendidos, são armazenadas de forma persistente em dois ficheiros de texto. A informação em cada ficheiros deve estar organizada por linha, em que os campos de dados são separados por ";", com o respetivo formato: '$COIN_VALUE; NUMBER_COINS$' (valor facial; número de moedas) e '$PRICE; SOLD_TICKETS; STATION_NAME$' (preço; número de bilhetes vendidos para a estação; nome da estação). Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

**Nota**: A inserção de informação através do teclado tem o seguinte critério: *i*) se não for premida nenhuma tecla num intervalo de cinco segundos o comando em curso é abortado; *ii*) quando o dado a introduzir é composto por mais que um dígito, são considerados apenas os últimos dígitos, a inserção realiza-se do dígito de maior peso para o de menor peso.

# 2 Arquitetura do Sistema

O sistema será implementado numa solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Port Expander LCD* (*PELCD*); *iii*) um módulo de interface com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*), designado por *Port Expander Ticket Dispenser* (*PETD*); *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*.
Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será emulado utilizando quatro interruptores e três *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

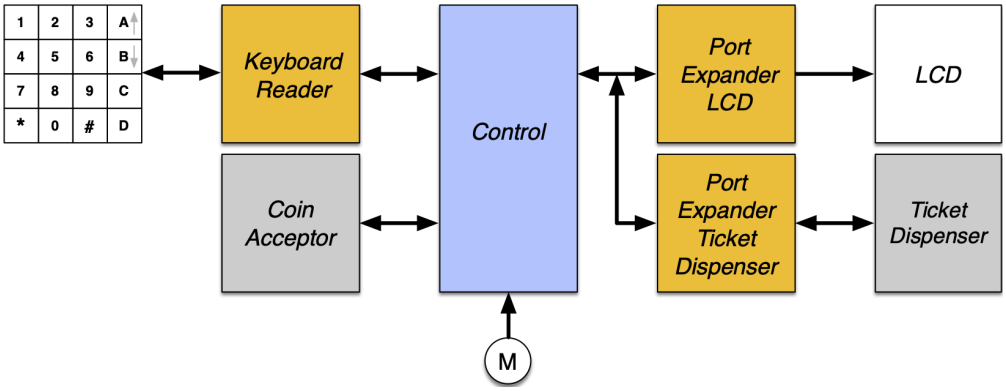



**Figura 2:** Arquitetura do sistema que implementa a máquina de venda de bilhetes (*Ticket Machine*)

O módulo *Keyboard Reader* é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos. O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *PELCD*. O *Ticket Dispenser* é atuado pelo módulo *Control*, através do módulo *PETD*. Por forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *PELCD* e *PETD* é realizada através de um protocolo série.

## 2.1 *Keyboard Reader*

O módulo *Keyboard Reader* é constituído por três blocos principais: *i*) o descodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*). Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

### 2.1.1 *Key Decode*

O bloco *Key Decode* deverá implementar um descodificador de um teclado matricial $4 \times 4$ por hardware, sendo constituído por três sub-blocos: *i*) um teclado matricial de $4 \times 4$; *ii*) o bloco *Key*

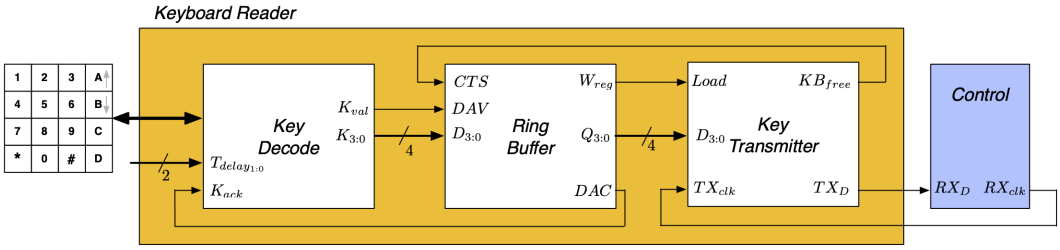
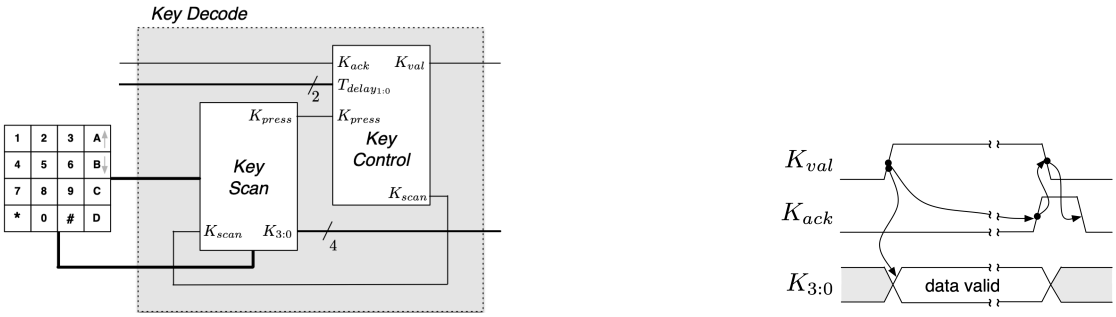



**Figura 3:** Diagrama de blocos do *Keyboard Reader*

*Scan*, responsável pelo varrimento do teclado; e *iii*) o bloco *Key Control*, que realiza o controlo do varrimento e o controlo de fluxo, conforme o diagrama de blocos representado na Figura 4a. O controlo de fluxo de saída do bloco *Key Decode* (para o bloco *Ring Buffer*) define que o sinal $K_{val}$ é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no barramento $K_{3:0}$. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal $K_{ack}$ for ativado e a tecla premida for libertada (numa primeira fase do projeto). O diagrama temporal do controlo de fluxo está representado na Figura 4b. Numa segunda fase do projeto, pretende-se adicionar um mecanismo de repetição de tecla (*auto repeat*); se após um determinado período, definido por $T_{delay_{1:0}}$ e conforme a Tabela 1, a tecla se mantiver premida gera novo código da tecla.




**(a)** Diagrama de blocos

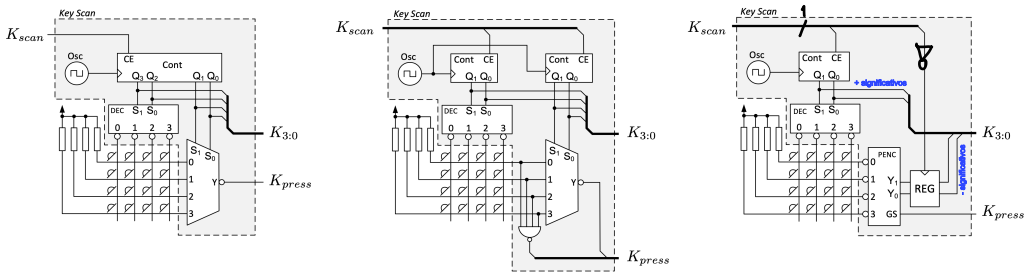
**(b)** Diagrama temporal

**Figura 4:** Bloco *Key Decode*

**Tabela 1:** Períodos do mecanismo de repetição

| $T_{delay_{1:0}}$ | período [ms] |
|---|---|
| 00 | 500 |
| 01 | 1000 |
| 10 | 1500 |
| 11 | 2000 |

O bloco *Key Scan* deverá ser implementado de acordo com um dos diagramas de blocos representados na Figura 5, enquanto o desenvolvimento e a implementação do bloco *Key Control* ficam como objeto de análise e estudo, devendo a sua arquitetura ser proposta pelos alunos.




**(a)** versão I **(b)** versão II **(c)** versão III

**Figura 5:** Diagrama de blocos do bloco *Key Scan*

### 2.1.2 *Ring Buffer*

O bloco *Ring Buffer* a desenvolver deverá ser uma estrutura de dados para armazenamento de teclas com disciplina $FIFO$ (*First In First Out*), com capacidade de armazenar até dezasseis (16) palavras de quatro (4) bits. A escrita de dados no *Ring Buffer* inicia-se com a ativação do sinal $DAV$ (*Data Available*) pelo sistema produtor, neste caso pelo *Key Decode*, indicando que tem dados para serem armazenados. Logo que tenha disponibilidade para armazenar informação, o *Ring Buffer* escreve os dados $D_{3:0}$ em memória. Concluída a escrita em memória ativa o sinal $DAC$ (*Data Accepted*) para informar o sistema produtor que os dados foram aceites. O sistema produtor mantém o sinal $DAV$ ativo até que $DAC$ seja ativado. O *Ring Buffer* só desativa $DAC$ depois de $DAV$ ter sido desativado. A implementação do *Ring Buffer* deverá ser baseada numa memória $RAM$ (*Random Access Memory*). O endereço de escrita/leitura, selecionado por $put\overline{get}$, deverá ser definido pelo bloco *Memory Address Control* ($MAC$) composto por dois registos, que contêm o endereço de escrita e leitura, designados por $putIndex$ e $getIndex$ respetivamente. O $MAC$ suporta assim ações de $incPut$ e $incGet$, gerando informação se a estrutura de dados está cheia (*Full*) ou se está vazia (*Empty*). O bloco *Ring Buffer* procede à entrega de dados à entidade consumidora, sempre que esta indique que está disponível para receber, através do sinal *Clear To Send* ($CTS$). Na Figura 6 é apresentado o diagrama de blocos para uma estrutura do bloco *Ring Buffer*.

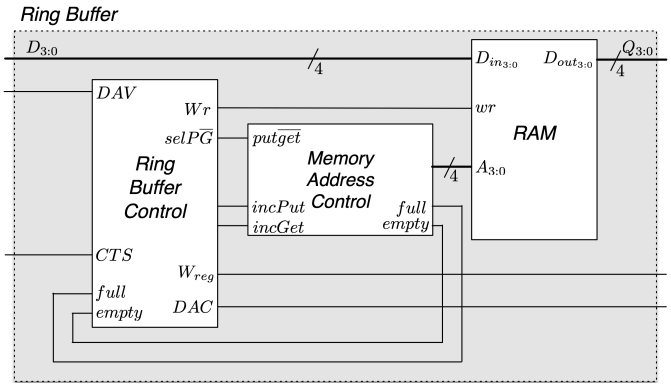



**Figura 6:** Diagrama de blocos do *Ring Buffer*

### 2.1.3 *Key Transmitter*

O bloco *Key Transmitter* do *Keyboard Reader* é responsável pela interação com o sistema consumidor, neste caso o módulo *Control*. O bloco *Key Transmitter* a desenvolver corresponderá a uma estrutura de transmissão série, com capacidade para armazenar e transmitir uma palavra de quatro bits. O *Key Transmitter* indica que está disponível para armazenar dados através do sinal $KB_{free}$. Assim, nesta situação o sistema produtor pode ativar o sinal *Load* para registar os dados na sua memória interna. Concluída a escrita na memória interna, desativa o sinal $KB_{free}$ para informar o sistema produtor que os dados foram aceites. Este bloco quando tem dados para transmitir, sinaliza o módulo *Control* através duma transição descendente do sinal $TX_D$, enquanto o sinal $TX_{clk}$ está no valor lógico zero. A cada transição ascendente do sinal $TX_{clk}$ o bloco *Key Transmitter* coloca o sinal $TX_D$ com o valor lógico necessário em cada instante, de acordo com o protocolo representado na Figura 7. Para que uma nova palavra possa ser armazenada neste bloco será necessário que o módulo *Control* tenha recebido todos os bits, e que este ative o sinal $KB_{free}$. A implementação do bloco *Key Transmitter* fica como objeto de análise e estudo, devendo a sua arquitetura ser proposta pelos alunos.

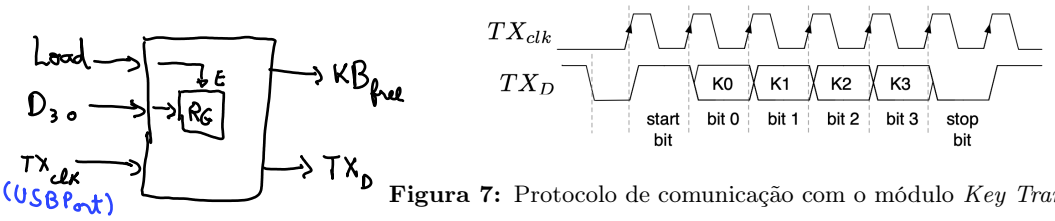
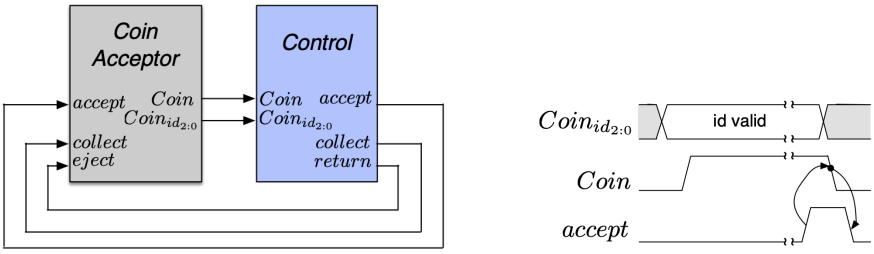



**Figura 7:** Protocolo de comunicação com o módulo *Key Transmitter*

## 2.2 *Coin Acceptor*

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, o valor monetário da moeda inserida é codificado em $Coin_{id_{2:0}}$, conforme a codificação apresentada na Tabela 2. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 8b. O *Control* pode devolver as moedas inseridas no moedeiro através da ativação do sinal *eject* durante dois segundos, ou recolher as moedas presentes no moedeiro através da ativação do sinal *collect* durante dois segundos.




**(a)** Diagrama de blocos **(b)** Diagrama temporal

**Figura 8:** Bloco *Coin Acceptor*

**Tabela 2:** Codificação do valor facial das moedas

| Moeda [€] | Codificação $[Cid_2Cid_1Cid_0]$ |
|---|---|
| 0,05 | 000 |
| 0,10 | 001 |
| 0,20 | 010 |
| 0,50 | 011 |
| 1,00 | 100 |
| 2,00 | 101 |

## 2.3 *Port Expander LCD*

O módulo *Port Expander LCD* (*PELCD*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 9.

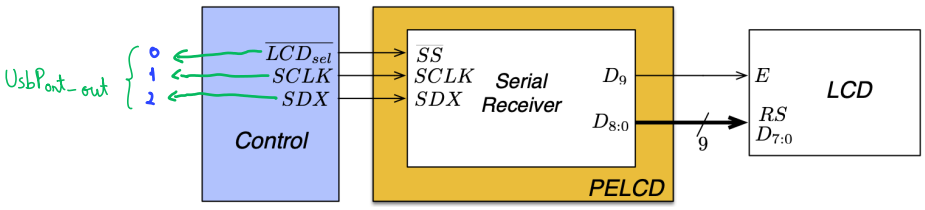



**Figura 9:** Diagrama de blocos do módulo *Port Expander LCD* (*PELCD*)

O módulo *PELCD* recebe em série uma mensagem constituída por dez (10) bits de informação. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura 10, em que o bit *RS* é o primeiro bit de informação e indica se a mensagem é de controlo ou dados. Os seguintes oito (8) bits contêm os dados a entregar ao *LCD*. O último bit contém o sinal de *E* responsável por registar a informação no *LCD*.

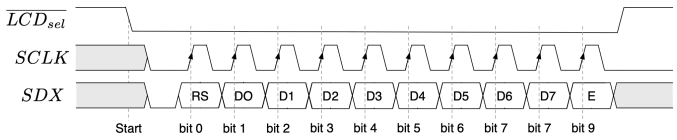



**Figura 10:** Protocolo de comunicação com o módulo *Port Expander LCD* (*PELCD*)

O emissor, realizado em software, quando pretende enviar uma trama para o módulo *PELCD* promove uma condição de início de trama (*Start*), que corresponde a uma transição descendente na linha ($\overline{LCD_{sel}}$). Após a condição de início, o módulo *PELCD* armazena os bits de dados da trama nas transições ascendentes do sinal $SCLK$.

O bloco *Serial Receiver* do módulo *PELCD* é constituído por dois blocos principais: *i*) um registo de deslocamento; e *ii*) um registo de manutenção; designados por *Shift Register* e *Hold Register*

respetivamente. O bloco *Serial Receiver* deverá ser implementado com base no diagrama de blocos apresentado na Figura 11.

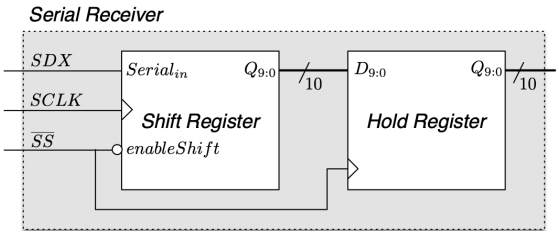



**Figura 11:** Diagrama de blocos do bloco *Serial Receiver*

## 2.4 *Port Expander Ticket Dispenser*

O módulo *Port Expander Ticket Dispenser* (*PETD*) implementa a interface com o mecanismo de impressão de bilhetes (*Ticket Dispenser*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mecanismo, conforme representado na Figura 12.

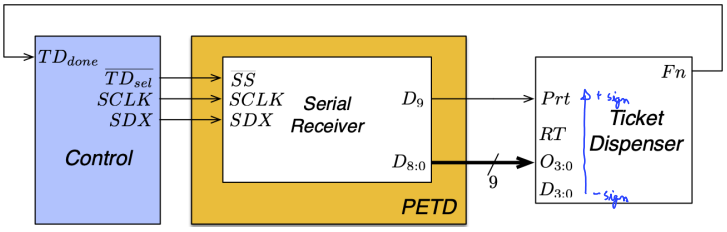



**Figura 12:** Diagrama de blocos do módulo *Port Expander Ticket Dispenser* (*PETD*)

O módulo *PETD* recebe em série uma mensagem composta por dez (10) bits de informação, segundo o protocolo de comunicação ilustrado na Figura 13. O primeiro bit de informação indica se o bilhete a imprimir é de ida ou ida/volta. Os quatro bits seguintes indicam a estação de origem, seguido da estação de destino por mais 4 bits de informação. O último bit de informação indica o início da impressão e disponibilização do bilhete. Só pode ser iniciada nova impressão quando o mecanismo de dispensa de bilhetes informar o *Control* que o processo terminou através do sinal *Fn*.

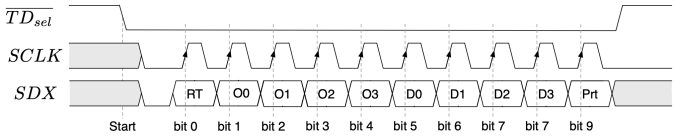



**Figura 13:** Protocolo de comunicação com o *Port Expander Ticket Dispenser* (*PETD*)

O *Ticket Dispenser* recebe em 4 bits o código da estação de destino ($D_{3:0}$), noutros 4 bits o código da estação de origem ($O_{3:0}$) a imprimir no bilhete e ainda o bit *RT* que define o tipo de bilhete (ida ou ida/volta). O comando de impressão do bilhete com os códigos presentes em *D*, *O* e *RT* é realizado pela ativação do sinal de impressão (*Prt*). Em resposta, o *Ticket Dispenser*

ativa o sinal de término de execução ($Fn$) quando concluída a dispensa do bilhete. Os sinais $Fn$ e $Prt$ têm o comportamento descrito no diagrama temporal apresentado na Figura 14.

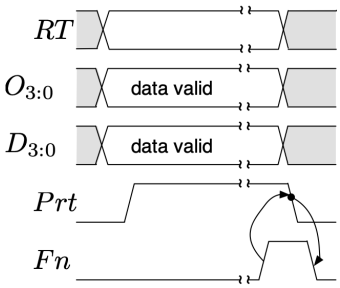



**Figura 14:** Protocolo de comunicação com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*)

## 2.5 *Control*

A implementação do módulo *Control* deverá ser realizada em software, usando a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 15. As assinaturas das principais funções e objetos a desenvolver são apresentadas nas próximas secções. As restantes são objeto de análise e decisão livre.

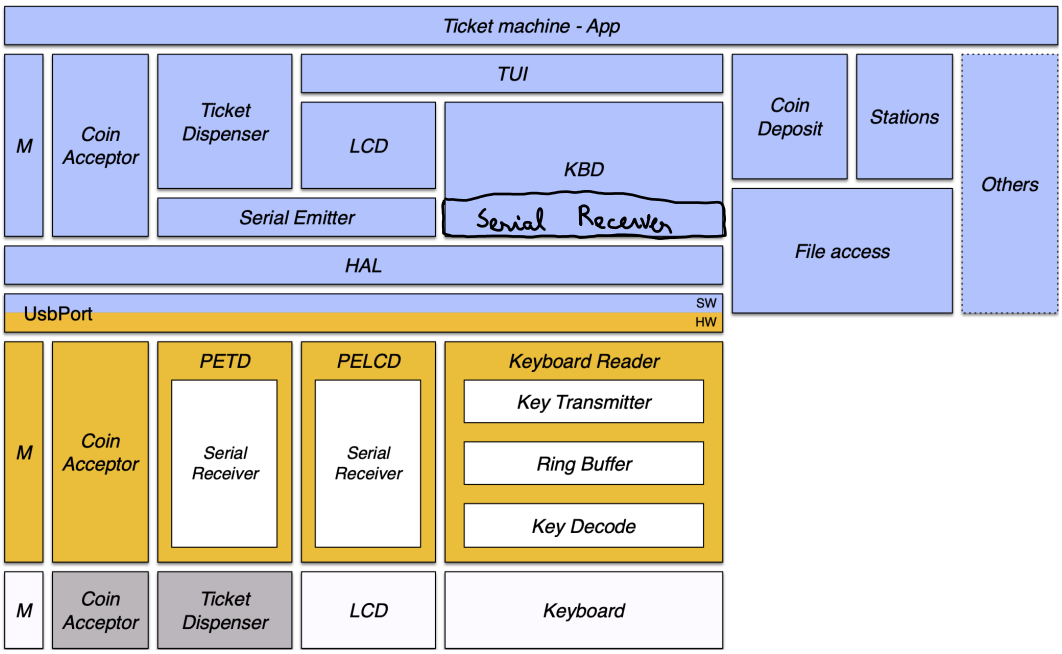



**Figura 15:** Diagrama lógico da Máquina de Venda de Bilhetes (*Ticket Machine*)

**Algoritmo 1:** HAL - Hardware Abstract Layer

```
object HAL {
// Inicia o objeto
fun init() ...

// Retorna 'true' se o bit definido pela mask esta com o valor logico '1' no UsbPort
fun isBit(mask: Int): Boolean ...

// Retorna os valores dos bits representados por mask presentes no UsbPort
fun readBits(mask: Int): Int ...

// Escreve nos bits representados por mask os valores dos bits correspondentes em value
fun writeBits(mask: Int, value: Int) ...

// Coloca os bits representados por mask no valor logico '1'
fun setBits(mask: Int) ...

// Coloca os bits representados por mask no valor logico '0'
fun clrBits(mask: Int) ...
}
```

**Algoritmo 2:** KBD - Keyboard

```
// Ler teclas. Funcoes retornam '0'..'9','A'..'D','#','*' ou NONE.
object KBD {
const val NONE = 0;

// Inicia a classe
fun init() ...

// Retorna de imediato a tecla premida ou NONE se nao ha tecla premida.
fun getKey(): Char ...

// Retorna a tecla premida, caso ocorra antes do 'timeout' (em milissegundos),
// ou NONE caso contrario.
fun waitKey(timeout: Long): Char ...
}
```

**Algoritmo 3:** LCD

```
// Escreve no LCD usando a interface a 8 bits.
object LCD {

// Dimensao do display.
const val LINES = 2, COLS = 16

// Escreve um byte de comando/dados no LCD em serie
private fun writeByteSerial(rs: Boolean, data: Int) ...

// Escreve um byte de comando/dados no LCD
private fun writeByte(rs: Boolean, data: Int) ...

// Escreve um comando no LCD
private fun writeCMD(data: Int) ...

// Escreve um dado no LCD
private fun writeDATA(data: Int) ...

// Envia a sequencia de iniciacao para comunicacao a 8 bits.
fun init() ...

// Escreve um carater na posicao corrente.
fun write(c: Char) ...
```

```
// Escreve uma string na posicao corrente.
fun write(text: String) ...

// Envia comando para posicionar cursor ('line':0..LINES-1 , 'column':0..COLS-1)
fun cursor(line: Int, column: Int) ...

// Envia comando para limpar o ecra e posicionar o cursor em (0,0)
fun clear() ...
}
```

**Algoritmo 4:** Serial Emitter

```
// Envia tramas para os diferentes modulos Serial Receiver.
object SerialEmitter {

enum class Peripheral {LCD, TICKET}

// Inicia a classe
fun init() ...

// Envia uma trama para o SerialReceiver
// identificado o periferico de destino em 'addr',
// os bits de dados em 'data'
// e em 'size' o numero de bits a enviar.
fun send(addr: Peripheral, data: Int) ...

// Retorna informacao se o periferico esta ocupado
fun isBusy() : Boolean ...
}
```

Retorna false depois de o bilhete ser retirado

**Algoritmo 5:** Ticket Dispenser

```
// Controla o mecanismo de dispensa de bilhetes.
object TicketDispenser {

// Inicia a classe, estabelecendo os valores iniciais.
fun init() ...

// Envia comando para dispensar um bilhete
fun activatePrintingTicket(roundTrip: Boolean, origin: Int, destination: Int) ... }
```

# 3 Calendarização

| Name | Begin date | End date |
|---|---|---|
| Ticket Machine | 26/02/2026 | 09/06/2026 |
| Hardware | 26/02/2026 | 22/05/2026 |
| Keyboard Reader | 26/02/2026 | 22/05/2026 |
| Key Decode – 1ª fase | 26/02/2026 | 04/03/2026 |
| Key Decode – 2ª fase | 18/05/2026 | 22/05/2026 |
| Key Transmitter | 20/04/2026 | 03/05/2026 |
| Ring Buffer | 04/05/2026 | 15/05/2026 |
| Validação | 18/05/2026 | 22/05/2026 |
| Serial LCD | 19/03/2026 | 01/04/2026 |
| Serial Receiver | 19/03/2026 | 25/03/2026 |
| Validação | 19/03/2026 | 01/04/2026 |
| Serial Ticket Dispenser | 13/04/2026 | 17/04/2026 |
| Serial Receiver | 13/04/2026 | 17/04/2026 |
| Validação | 13/04/2026 | 17/04/2026 |
| Testes e correção de erros | 18/05/2026 | 05/06/2026 |
| Software | 05/03/2026 | 05/06/2026 |
| HAL | 05/03/2026 | 11/03/2026 |
| KBD | 05/03/2026 | 18/03/2026 |
| Serial Receiver | 20/04/2026 | 03/05/2026 |
| Serial Emitter | 19/03/2026 | 25/03/2026 |
| LCD | 26/03/2026 | 01/04/2026 |
| Ticket Dispenser | 13/04/2026 | 17/04/2026 |
| TUI | 20/04/2026 | 24/04/2026 |
| File Access | 27/04/2026 | 02/05/2026 |
| Coin Deposit / Stations | 04/05/2026 | 08/05/2026 |
| APP | 11/05/2026 | 05/06/2026 |
| Testes e correção de erros | 18/05/2026 | 05/06/2026 |
| Validação (Sistema) | 18/05/2026 | 05/06/2026 |
| Relatório | 26/02/2026 | 09/06/2026 |
| Documentação | 26/02/2026 | 05/06/2026 |
| Avaliação Intercalar I | 01/04/2026 | 01/04/2026 |
| Avaliação Intercalar II | 18/05/2026 | 18/05/2026 |
| Entrega Relatório | 11/06/2026 | 11/06/2026 |
| Kick-off | 18/02/2026 | 04/03/2026 |
| Apresentação UC | 18/02/2026 | 24/02/2026 |
| Formação de Grupos | 18/02/2026 | 24/02/2026 |
| Descrição Comportamental | 18/02/2026 | 24/02/2026 |
| Apresentação Projeto | 18/02/2026 | 24/02/2026 |
| Kahoot! | 26/02/2026 | 04/03/2026 |
| Validação Projetos – Época Normal | 08/06/2026 | 12/06/2026 |
| Discussões Finais – Época Normal | 15/06/2026 | 26/06/2026 |
| Validação Projetos – Época Recurso | 15/06/2026 | 26/06/2026 |
| Discussões Finais – Época Recurso | 29/06/2026 | 10/07/2026 |

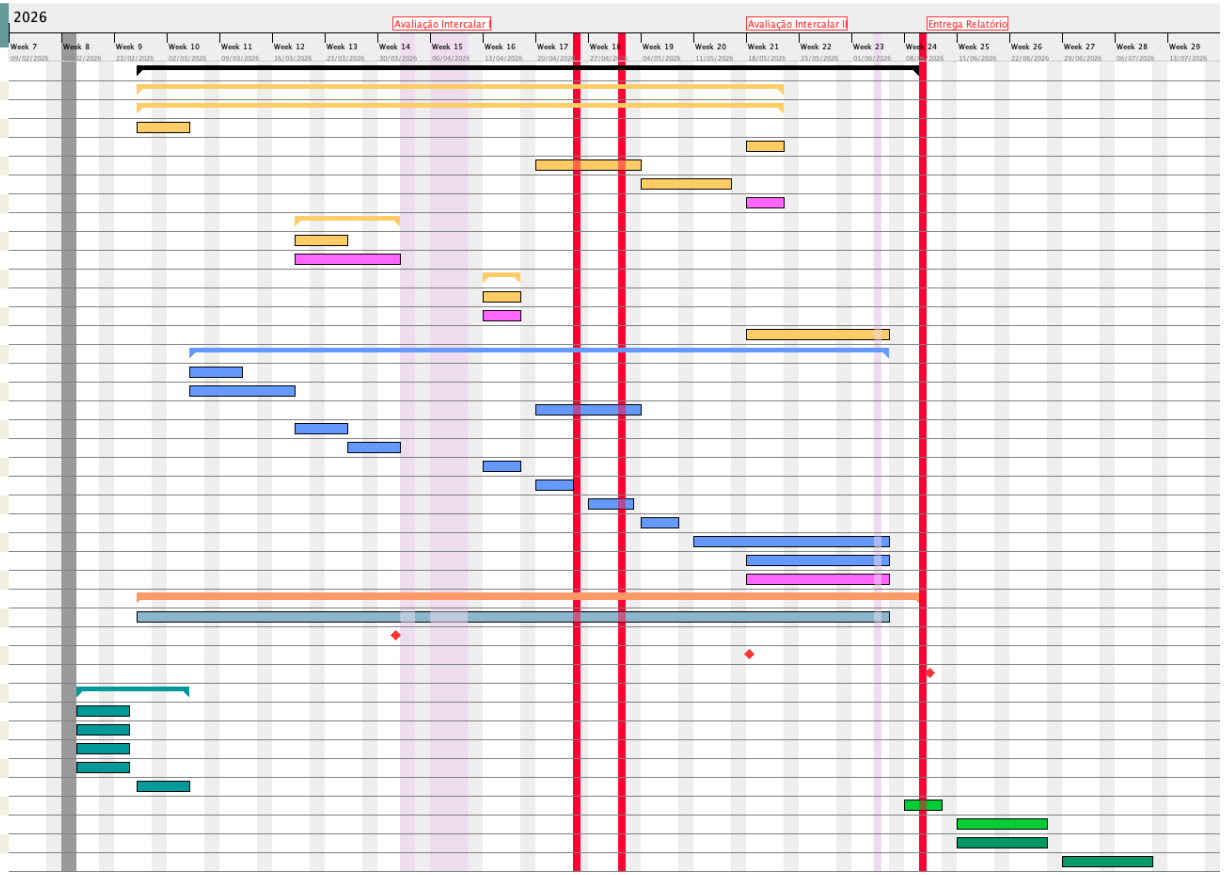



**Figura 16:** Diagrama de *Gantt* relativo à calendarização do projeto)